Monkey business arithmetic spectator pipeline

Kirill Kovlekov
Independent Researcher

November 2026

Auditable numerical computation in safety-critical and regulated systems.

---

1. Introduction

1.1. The framework

Every layered system eventually fails the same way. A bug in one layer rewrites another layer's work, and by the time the corruption surfaces, the origin is unrecoverable. The mitigation is almost always a convention: a serialization format, a protocol version, a sandbox boundary, a type annotation. Conventions work until they don't, and the failure mode is always the same - the layer that was supposed to be isolated *understood* something it was not supposed to understand, and used that understanding to change something it was not supposed to change.

But what if we could separate the algebra of calculating (mathematical operations) from the algebra of computing (weighing and balancing outputs) so that compute layers cannot interpret+run supervisory operations and supervisory layers cannot interpret+run compute operations. If this separation is algebraic rather than conventional, we can afford to set the architecture so that neither side has the semantics *or* the operation. This means no cross-level leaks.

This paper presents to you the **Monkey Business Arithmetic Spectator Pipeline**: a conjectural (until verified by a completed test build) framework for building an experimental 6-level arithmetic pipeline.

The paper will be divided into two parts: 1) Pedagogical – a gradual step-by-step exposition of the pipeline with definitions and explanations for easing yourself into a daring math domain; 2) Specs – the compact spec-sheet of the architecture.

1.2. Why you should care?

What if you had a combination of:

1. Differential debugging of long-latency anomalies (Deep Space Kraken, the Far Lands etc)

2. Compositional verification of layered arithmetic.

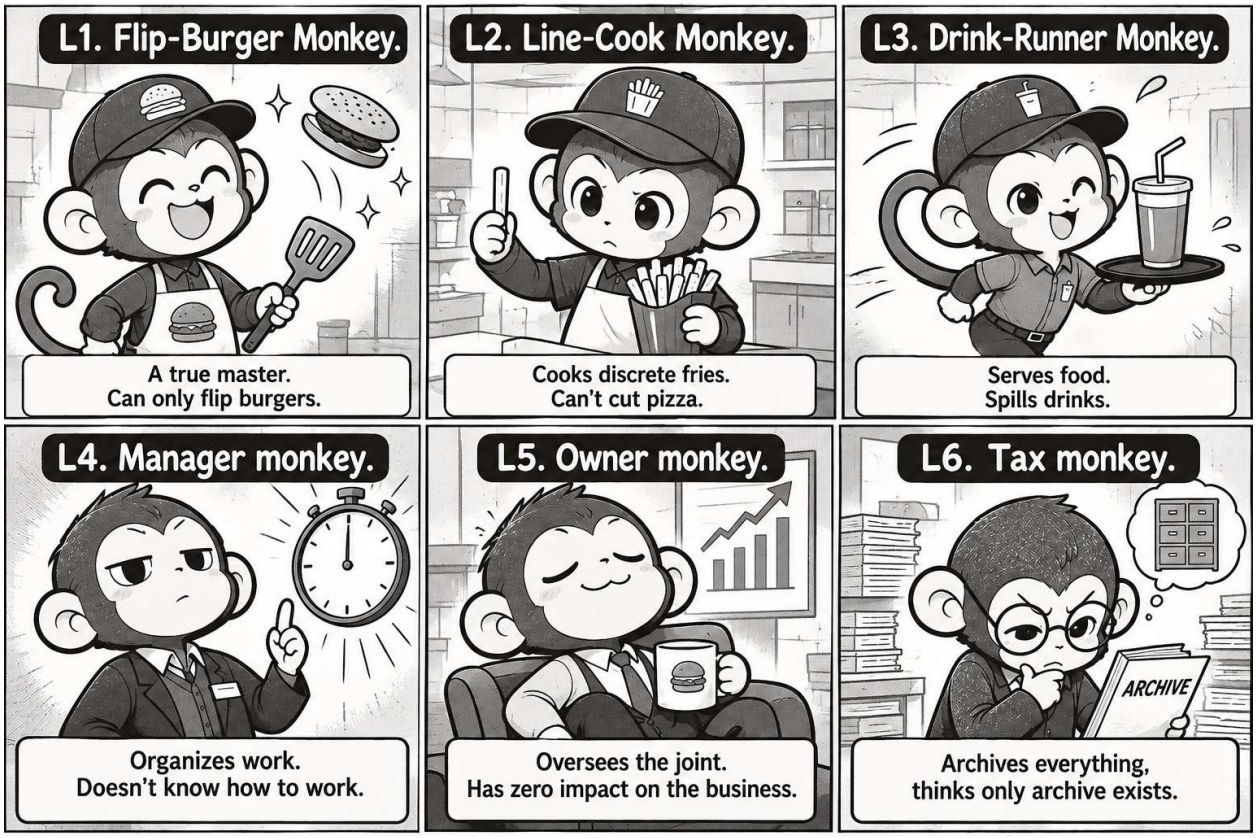
3. Capability-style security for computation.

4. Reproducibility by construction.

5. Heterogeneous offload with end-to-end visibility.

Fitting them together in a single pipeline is sure to feature some monkey business.

1.3. Why monkey business? (A parable)



Imagine that there exists a fast food joint that is run entirely by monkeys. It actually works despite that every monkey is somehow completely inept:

Worker monkeys:

1. **Flip-Burger Monkey.** The true master of flipping burgers. Can only flip burgers.
2. **Line-Cook Monkey.** Can cook everything discrete like French fries. Doesn't understand how to operate the soda machine.

3. **Drink-Runner Monkey.** Serves food by floating around the joint, pours drinks, but spills a lot.

Supervisor monkeys:

4. **Manager monkey.** Tells every worker what and when to do, but doesn't know how work is actually done, and can only be satisfied when the day ends.
5. **Owner monkey.** Carefully watches the joint run, but has zero impact on the business. Communicates with investors.
6. **Tax monkey:** Archives every result in a ledger, thinks that only records matter.

And you are the investor monkey!

1.4. What are monkeys?

We are not talking about monkeys, of course.

Monkey Business Pipeline (MBP for short) operates as a stack of 6 divided environments, where each one represents a level (L1-L6) in a nested hierarchy.

The idea is simple – we take different arithmetical/logical tasks and order them in a layered architecture (algebraically separated) with a specific unified design, that is both sequential and is a self-similar generative process.

Calculational environment (worker monkeys) are meant for doing level-relative math calculations. Binary (Flip-Burger Monkey), Integer (Line-Cook Monkey) and Float (Drink-Runner Monkey) maths, to be exact.

Computational environment (supervisor monkeys) are meant for conducting the parallel calculations (Manager Monkey), reading output and UX (Owner Monkey) and logging/memory (Tax Monkey).

Levels:

The whole architecture will be realized as a single nested algebra, levels wouldn't be able to interpret each other's output (only copy/translate packages). Cross-level communication will operate under a very restrictive trust chain (hence the pipeline).

L1: Binary environment (N operations)

L2: Integer environment (Z operations)

L3: Float environment (Q operations)

L4: Complex environment (simulates a complex field to algebraically balance L1-L3)

L5: UC environment (UC means Ultra-complex and features a familiar algebra rendered for the L5 purposes)

L6: UC+ environment (An extended UC algebra with an overflow bin)

Author's note: If you are interested in how environments relate to the N-Z-Q – please read the "6 geometric freedoms" paper first: https://github.com/Bitriada/poetica-mathematica/blob/main/3_Standalone_papers/6_geometric_freedoms/Generative_Construction_of_6_Geometric_Freedoms.pdf

Scales:

Each level will be represented by a triadic scale (osi):

Object (o) – a minimal unit. The main arithmetical object of the level.

Structure (s) – a unit of units. The data structure of the level.

Instance (i) – an environment of unit of units. A closed algebraic instance, operating under a pre-determined ruleset/moveset.

This distinction is important because for each given level of the pipeline there is a limited operation signature that is scale-indexed.

1.5. Disclaimer

To avoid cognitive dissonance - focus on the architecture, not solutions.

The MBP is a technical specification for people who are willing and capable enough to test it. It is not a textbook case study, so, if you find math that doesn't fit, this may be a legitimate window of opportunity for optimization.

2. Flip-Burger Monkey (L1)

2.1. 8-bit string

For the register of this level we take an 8-bit string - a byte-sized minimal data structure that will become the foundation for 256-base design.

There are 2 main reasons for this:

1) 8-bits are productive, minimal and byte-aligned
2) Further levels will double the bit-size each round (6th level will be 256 bit), so, minimalism is a virtue;

And 3 more:

3) 256-base is mathematically rich (e.g. scale/division);
4) 8-bits allow for faster reading;
5) We won't need more anyway for this level. ((ℕ)^256 worth of binary strings/flags should be enough)

2.2. A tape

This level is meant for the **natural** domain of **numbers**. Here we treat them as a **tally** counter for real, existing, discrete items. I.e. this level is meant for tallying distinctive processes, objects and classes that would be live within the runtime.

The best visual representation for how this level operates is **a tape.**

A tape is sequential, one-dimensional medium that writes out a single string of data. No rows, no columns, only length.

Now picture a single tape that records binary strings in a sequence:

Scale:	**Identity:**
o	A bit
s	A binary string
i	The tape

The design of this level was only inspired by a Turing's machine, because the operation set is pretty daft as you will see.

2.3. Complete operation set

The complete operation set:

Scale	Pair	Operation	Target
o	`o+1 ; o-1`	Tally up / Tally down	bit
s	`s+1 ; s-1`	Open / Close	binary string
i	`i+1 ; i-1`	Boot / Export	tape

That's it.

2 operations of: +1 and -1, that apply at three scales and complete a moveset for a discrete geometry of points and strings.

Author's note: if you find L1-L3 algebras too novel, please, refer to "6 geometric freedoms" paper mentioned in introduction.

2.4. The cycle

Binary environment instance is completed in a single round of 3 stages:

I) i+1
II) A: [s+1; o+1 OR o-1; s-1]; B: [s+1; o+1 OR o-1; s-1]; etc
III) i-1

i+1 is an operation that boots up the binary environment by creating a blank tape.

Strings A, B etc are formed sequentially. s+1 opens a string, tally goes up or down, s-1 closes the string and the cycle is repeated as many times as needed.

Then i-1 exports the tape as a package (a single tape of however much binary string were created) and terminates the instance.

2.5. But why?

The algebra of +1/-1 is the extremely compacted for one purpose only: create a calculational environment where you can only count up or down for however many things that you want to count. This means that no negative integers or other possibilities for a miscalculation would be possible.

This primes the trust chain of the whole pipeline in a most dumb way possible – you can trust L1 to never be able to do anything beyond what little it can do in a 1D operational space. That's the crux of the whole security model of the pipeline.

Ready for 2D?

3. Line-Cook Monkey (L2)

3.1. 2x 8-bit strings (16-bit environment)

For this environment we will have to double the aggregate data width by advancing a rank of our structure from scalar to a matrix.

256*256 = 65536

But what kind of operations require such an upgrade?

3.2. A punch card

L1 produced a tape: an instance of x binary strings (1 byte for each). By the end of L2 we will have a data structure inspired by a punch card: an instance of x integer cells

This structure is necessary for representing **abstract numbers**.

As natural numbers were keeping tallies, the abstract numbers will keep counters for the tallies.

1+1+1+1+1 is a tally worth of 5x “ones”.

But 5 is a counter without the tally.

3.3. Complete operation set

If we string together multiple tally up “+1” operations we get addition as an emergent operation: “+a”.

+1+1+1+1+1 = +5a

Compared to L1 this is an explosion of complexity, but we are not done yet.

+a +a +a +a +a = 5*a

This seems trivial, but from perspective of L1 this is what distinguishes Flip-Burger Monkey from a Line-Cook Monkey.

Scale	Pair	Operation	Target
		Turn 1	
o	`o+a; o-a`	add/remove	counter

Scale	Pair	Operation	Target
		Turn 1	
s	`s+a; s-a`	open/close	Cell/Set
i	`i+a; i-a`	boot/relay	Card
		Turn 2	
o	`o*a ; o/a`	multiply/partition	multiplier
s	`s*a ; s/a`	open/close	Cell/Set
i	`i*a ; i/a`	expand/export	Card

In other words, it's the same scale-indexed operation set, doubled from +1; -1 to +a; -a; *a; /a which enable abstract operations for counting in sets of numbers and subsequently allowing for negative number line to emerge.

3.4. The cycle

The cycle spans 1 round, but now divided into 2 turns that secure the linearity of the algorithm (no interleaving).

The sequence continues:

Turn 1:

I) Boot (`i+a`):

L2 boots the instance (a blank card) and receives a package from L1. The tape we receive is going to be cut by the next step.

II) Coefficient:

`s+a` creates a coefficient axis of cells.

`o+a` or `o-a` writes the tally into the cell as an integer;

`s-a` closes the cell;

Repeat for every tally.

III) Relay (`i-a`).

When the tape is exhausted and the axis has all its coefficients, `i-a` closes the axis. At the end we receive a 1D set of integers translated from the L1 package into L2.

Turn 2:

IV) Expand (`i*a`)

`i*a` receives the closed coefficient axis and establishes the second dimension. The structure becomes a full-fledged 2D card:

1) The coefficient axis made up of the Turn 1's integer set.
2) Multiplicator/divisor integer line representing 256-base with 256 cells.

The multiplier axis is a reflection of Z, the integer line itself, held against the coefficient axis.

V) Multiplier:

o∗a and o/a populate the cells. This produces the matrix of multiplicators/divisors. Each integer is no longer just a tally, but a counter for tallies that carries scaling info in the emergent cells.

Turn 2 runs the length of the integer line: 256 positions, one multiplier cell per position. For each position:

`s*a` opens a multiplier cell in the second column; `s/a` closes it.

`o*a` or `o/a` are used to populate the cells. This produces the matrix of multiplication/division product. This matrix is a primitive way to analyze the numbers without importing any semantics beyond what was already there.

Turn 1 translated the L1 tallies into L2 counters and then Turn 2 essentially provides a behavioral matrix for the numbers under multiplication/division.

This makes the Line-Cook Monkey data instantly not readable by the Flip-Burger Monkey, which seals the semantic layer L2 operates on.

Also, the cells can carry divisibility information latently, without the need to define what parity is. A cell that resolves under /2 or /3 is marked by that resolution. This information isn't readable by the L2 (no operations for it), but for later levels it can be useful. Also, the whole /2 string can be used to later order the numbers with odd/even. This is the first time the generative grammar behind this pipeline shows its head.

Well, this arithmetic pipeline is based on hyper-operation sequence treated as a constructive process which creates different numerical properties solely through operation/structure cooperation.

Anyway, once the Line-Cook Monkey is done "preparing the meal" (by analyzing integers) it can be handed off to the next Monkey.

VI) Export card (i/a)

`i/a` closes the multiplier dimension and releases the card as a package.

Example:

To visualize the output of Turn 2, here is a reduced 5x5 rendition of the card.

The base set is **a = [1, 2, 3, 4, 5]**.
The rows represent **x** (the divisor axis), and the columns represent **y** (the multiplier axis).
The operation is **M[x, y] = (a / x) * y**.

x \ y	*1	*2	*3	*4	*5
/1	[1, 2, 3, 4, 5]	[2, 4, 6, 8, 10]	[3, 6, 9, 12, 15]	[4, 8, 12, 16, 20]	[5, 10, 15, 20, 25]
/2	[1/2, 1, 3/2, 2, 5/2]	[1, 2, 3, 4, 5]	[3/2, 3, 9/2, 6, 15/2]	[2, 4, 6, 8, 10]	[5/2, 5, 15/2, 10, 25/2]
/3	[1/3, 2/3, 1, 4/3, 5/3]	[2/3, 4/3, 2, 8/3, 10/3]	[1, 2, 3, 4, 5]	[4/3, 8/3, 4, 16/3, 20/3]	[5/3, 10/3, 5, 20/3, 25/3]
/4	[1/4, 1/2, 3/4, 1, 5/4]	[1/2, 1, 3/2, 2, 5/2]	[3/4, 3/2, 9/4, 3, 15/4]	[1, 2, 3, 4, 5]	[5/4, 5/2, 15/4, 5, 25/4]
/5	[1/5, 2/5, 3/5, 4/5, 1]	[2/5, 4/5, 6/5, 8/5, 2]	[3/5, 6/5, 9/5, 12/5, 3]	[4/5, 8/5, 12/5, 16/5, 4]	[1, 2, 3, 4, 5]

The optimal way to treat this package would be as a rendition of Z as base-256 math, where the integer domain travels with the card in a form of a multiplier

layer. It might lead to some exciting engineering decisions, but the structure of it is pretty simple: the coefficient as a sequence of digits, and the multiplier as the radix layer of basic analytical information. In other words, technically it is not a mixed-radix system: but it can be read as such further down the line. (but not on this layer, for which we intentionally omitted the notion of an address space at all)

This being said: "/" is a terminal operation of the whole level because it causes a crisis.

This is as far as the structural analogy goes: L2 doesn't possess the semantics to operate in a higher-dimensional expression language such as ZFC.

3.5. Unresolvable division

Look closely at the `/2` and `/3` rows. You will find cells like `3/2` and `5/3`. The matrix successfully generated the coordinates for these values - it knows they exist between the integers - but L2 has no operation to resolve them into a single number. They remain structural ratios trapped on the card.

Formal analogy (with caveat): The L2 card can be read as a finite, partial Cayley-style table of the multiplicative monoid over a finite base set: the row operator is division by x, the column operator is multiplication by y, and each cell is their composite. Unlike a Cayley table of a group, which is square and total, a monoid table may be rectangular and partial; the card is of the second kind. The partial cells - $3/2$, $5/3$, etc. - are exactly the cells where division escapes the base set Z into $Q \setminus Z$.

In other words, we have division that doesn't resolve with integers.

This can be treated as an **in-built overflow mechanic that prevents the overflow by causing it**, then passing the ball to the next level. The record of those numbers would be available for L3 to resolve.

And that's why L3 is about creating an address space.

4. Drink-Runner Monkey (L3)

4.1. Why 32-bit

We can use 32 bits here because the nesting deepens.

- L1 was depth-1: one tape of 8-bit strings.
- L2 was depth-2: one integer set represented as a card of two 8-bit strings. Aggregate bit-width of 16-bits.
- L3 will be depth-3: 2 cards, 4 strings – 32 bits total.

Depth here means a nesting of the 256-base structure.

4.2. A loom

L2 was inspired by a punch card, so, it is only natural that L3 is inspired by the Jacquard loom. The metaphor became progressively more physical: Turing's tape as theoretical, Babbage's punch card as mechanical and Jacquard's loom as an industrial implementation kind of progression – not exact, but works as an illustration for Worker Monkeys.

The idea of a loom is a neat visual metaphor for how integers are prepared to face the float environment.

Imagine a float number as $a.bbbb$. Each integer is an address, while every interval between them populated by ratios is a cycle. In other words, the instance of L3 is a necklace of rings weaved by an even more elaborate operation set, that is also emergent from the previous one. Notice that the number of turns follows a strict 1-2-4 progression, mirrored by the operations: 6-12-24 progression. (If you wish to read more on what combinatorial object was used in the design of the progression, please refer to the "Generative Integer Fractal from minimal rules" paper: https://github.com/Bitriada/poetica-mathematica/blob/main/3_Standalone_papers/Arithmetic_Bitriada/Generative_Integer_Fractal_from_minimal_rules.pdf)

4.3. Complete operation set

Scale:	Pair:	Operation:	Target:
Turn 1. Address grid			
o	o+b; o-b	Move forward/backward	Position
s	s+b; s-b	Map/Unmap	Address
i	i+b; i-b	Boot/Relay	Register
Turn 2. Address space			
o	o*b; o/b	Dial up/down	Position
s	s*b; s/b	Map/Unmap	Address
i	i*b; i/b	Boot/Relay	Register
Turn 3. Complex overflow space			
o	o^b; o√b	Project/Enfold	Position
s	s^b; s√b	Map/Unmap	Address
i	i^b; i√b	Boot/Relay	Register
Turn 4. Recovery space			
o	o|b|; o[[b]]	Extract/Obfuscate	Position
s	s|b|; s[[b]]	Extract/Obfuscate	Address
i	i|b|; i[[b]]	Extract/Obfuscate	Register

In L3, the operations from L2 (`+a; -a; *a; /a`) are retained, but they are re-used unber the *b* interpretation. If these operation are already perfect – no need to substitute them like we did with `+1; -1`.

Repeated use of `*b` naturally produces a new operation: `^b` (exponentiation), with `√b` being its inverse. So, far so good. More on extraction and obfuscation later.

4.4. The cycle

The sequential unfolding of the float space happens across four turns. The first two turns essentially read the L2 package and translate it into the L3 float language.

Turn 1: The Address Grid

Geometric Counterpart: Setting the the integer line.

Cycle:

1. **Boot:** `i+b` boots the signed register (uses discrete integers from L2).
2. **Map:** `s+b` maps an address.
3. **Operate:** `o+b` or `o-b` moves the position forward or backward along the integer line until it finds the number it is looking for.
4. **Unmap:** `s-b` closes the position.
5. **Relay:** `i-b` closes the signed register and relays the grid.
   Output: A fixed set of addressable integers.

Turn 2: The Address Space

Geometric Counterpart: Rendering the intervals between integers.
Cycle:

1. **Boot:** `i*b` boots the unsigned register (uses ratios from L2)
2. **Map:** `s*b` maps an address within the grid from Turn 1.
3. **Operate:** `o*b` or `o/b` dials the position up or down. Geometrically, this is modulo multiplication/division: it shifts the position around the ring (`+b`/`-b` within a cycle) or distributes the ring across cycles (`/b`). This establishes the exact position of the previously unrenderable counter as an address.
4. **Unmap:** `s/b` closes the position.
5. **Relay:** `i/b` closes the unsigned register and relays the space.
   Output: The populated address space.

With these two turns the necklace of rings is established and the L2 package is translated into L3.

The next two turns commence further (risky) operations.

Turn 3: The Complex Overflow Space

Geometric Counterpart: Volume expansion (Higher dimensions) and Complexification (Curve).
Cycle:

1. **Boot:** `i^b` boots the complex overflow register.
2. **Map:** `s^b` maps the address to be lifted.

3. **Operate:** o^b projects the value via exponentiation (dimensional lift). This calculates the *volume* of the number. Or o√b, which enfolds the number via rooting.
4. **Unmap:** s√b closes the address.
5. **Relay:** i√b closes the register and relays the lifted/returned tensor.
   Output: The values have been sent to the complex plane (L4) and returned.

Turn 4: The Recovery Space

Please read the "6 Geometric Freedoms" paper to learn more on what exactly we are doing here.

Nevertheless, sending the number into higher dimensions means the L3 float environment can no longer render its sign, because the number now represents an unfathomable volume. On the other hand, sending the number into a complex field of L4 yields us a number that is enriched with a complex phase.

These are exact reasons we can't use either output at L3. These numbers are no longer containable on par with the others. But there is a workaround. We can catch the impostor numbers and mark them for the potential risks they hold, minimizing the damage.

Cycle:

1. **Boot:** i|b| boots the recovery register.
2. **Map:** s|b| maps the address of the returned values.
3. **Operate:** o|b| extracts the magnitude, catching the higher-dimensional imposter and separates it from the safe addresses. o[[b]] obfuscates the phase. Because the number was sent to the complex plane via rooting, so, we mark the number as potentially having a complex phase.
4. **Unmap:** s[[b]] closes the address.
5. **Relay (Terminal):** i[[b]] closes the register, releases the tensor, and terminates the instance.
   Output: A certified or an uncertified float number tensor.

4.5. Semantic security before complex field

The Drink-Runner Monkey utilizes maximum computational power of the real numbers, but it spills half-way through into the complex field. So, the float environment is a semantically risky domain, that's why we use a structural analogue of the **taint markers**, to filter the outputs automatically.

This marks the point where Worker Monkeys end their duty and handoff the unfolded mathematical domain to the Supervisor Monkeys that take numbers to even more strange places.

5. Manager Monkey (L4)

Status. L4 is the pivot of the paper, and it is the first section whose mathematics is *cited, not derived*. The algebras are candidates; the architecture is what this section argues for. Concrete specifications are in §8.

5.1. Algebraic Manager

Each worker produced numbers that can be translated into a concrete geometry:

Level	Monkey	Geometry	Package
L1	Flip-Burger	Point	A tape - presence and absence
L2	Line-Cook	Line	A card - cells on an integer line
L3	Drink-Runner	Figure	A tensor - cycles traced between integers

So, that's the output that Manager will need to read and balance out. The idea is to use complex mathematics to model L1-L3 as a progression of complex objects: a point, a line and a figure – which would be later algebraically collapsed into an image of a complex field by the interface algebra of L4.

5.2. Structure of the Manager

The progression of bit-width leads us to 64 aggregate bits. While the concrete construction is pending test builds – we do possess a natural scaling unit for the Manager already available: **dimensions**. The final construction can be cleanly mapped to the Cayley-Dickson checkpoints:

Checkpoint	Algebra	Role
4	$\mathbb{H}$	the weighing primitive

Checkpoint	Algebra	Role
8	𝕆	one render step
64	𝕊 ⊗ UC	the highest render target

The idea is not to traverse the complex plane as a measurement would do, but to segment it into operable chunks (CD checkpoints).

5.3. Initial operation set

The complex plane introduces four fundamental operations:

Operation	Symbol	Action	What the Manager is doing
Rotation	R(θ)	multiply by `e^(iθ)`	attending to the field from a different angle
Conjugation	C	reflect across real axis	reading the field backwards
Modulus	M	collapse to magnitude `|z|`	weighing without caring which way it points
Phase Fixing	P(φ)	project onto the ray at angle φ	aligning the field to one direction

Contextual progression of operations: L1, L2, L3 can be seen collectively as a single context that began with 2 primitive operations. So, you can imagine that all the volume of L1-3 was a volume of a single basic context. Well, L4 appears to be a context of its own that begins with 4 primitives.

I think that the complex operations should multiply in quantity, but, in theory, stay qualitatively within the same 4 families. The exact emergent progression along the CD dimensions needs to be crunched; what is fixed is the qualitative family structure - L4's operations sit at the level of a complex field, not at the level of a scalar or a card.

5.4. The Cycle

The Complex algebra `ℂ[τ]/(τ²)` with a nilpotent trace `τ` is the minimal candidate algebra that can render three lower geometries as a complex field.

An important note: the τ scaffolding is internal and will be explained in L5; the Manager exposes only the complex value, because it is a representation of $\mathbb{C}$.

The three worker algebras can be rendered as **parallel cuts** from a common ambient, each with its own ideal.

The idea is to eventually build a sphere but all we can manage is to imagine that complex field is unwrapping from a single point. This way the sequence can be guided under these conditions:

Stage	Imaginary unit	Rotation generator	What the Manager sees
Point	$i_1^2 = i_1$ (idempotent)	$\theta_1^2 = \theta_1$ (idempotent)	direction and rotation collapsed
Line	$i_2 = \pm i_2$ (sign-ambiguous)	$\theta_2^2 = -\theta_2$	direction emerges
Figure	$[[i_3]]$ (present, no value)	$\theta_3^4 = \theta_3$	rotation is genuine
Field	$i_4^2 = -1$	$\theta_4 = \tau$ (nilpotent trace)	the three partial views cohere

1) For the first step we look at a single point. 1D is how a sphere is seen from infinitely far. $i_1^2 = i_1$: applying the direction operator twice gives the same direction. No change.
2) Then we look at a sphere from a 2D, but the view we get is of a flat pancake from the side, so, it looks just like a line. And we can tell left from right! At the line stage, $i_2 = \pm i_2$: applying the operator gives two possible answers depending on branch. The line can be traversed either way and the operator can't tell which.
3) Now we look at the pancake from the top. This way we can no longer tell a sign, because nobody introduced a canonical embedding or any kind of measure. So, no matter how you look at it – the pancake is still a pancake. Direction collapses, i_3 is present as a symbol but carries no value: direction around a closed loop is everywhere locally defined but globally untrackable.
4) This is the step where Manager chokes to produce a sphere, because complex algebra is two-dimensional. So, that's how a complex field is completed as a mess of three different POVs. $i_4^2 = -1$: applying the

direction operator twice gives negation. A 180° rotation. The direction operator resolves into the familiar rotation.

The subscript is the stage. It is one imaginary unit, seen at four stages of the Manager's progressive reading. The equations $i_1^2 = i_1$, $i_2 = \pm i_2$, $[[i_3]]$, $i_4^2 = -1$ all say the same thing in different ways: at that stage, the unit cannot yet resolve the phase it needs to resolve. The fourth stage resolves it.

The closure. The four stages sum to the null origin: $i_1^2 + i_2^2 + i_3^2 + 1 = 0$. The three worker perspectives plus the origin balance, and the field-stage unit arrives with $i_4^2 = -1$. This is the Manager's act of measurement (closure) - the point → line → figure → field circuit closes back onto the point.

So, to own the joint we have to go 3D.

6. Owner Monkey

The Owner is the overseer position: the rung from which the pipeline becomes legible as a single nested algebra.

Given that L1-L3 operated in a structural 1D domain (a single line of operations), L4 operated in 2D domain (families of operations under Cailey-Dickson scaling) – this progression implies that L5 must be structurally 3D (whatever this means). The only problem is that 3D algebra problem is resolved only with quaternions.

So, what we can do is invent a 3rd algebraic dimension of context-traversal audit. When a real number is lifted into a complex plane – then it is hailed back the number passes a filter that destroys complex data and returns a zero. This implies that there is a nilpotent filter between real and complex computational contexts and we need a trace dimension to trace the paid fare that this traversal takes.

6.1. Solution

Let's take a number $a + bi + \tau + \tau i$.

If τ anticommutes with i and $\tau^2 = -1$, the expression is a quaternion.

If τ is nilpotent ($\tau^2 = 0$), it is a Study number - the algebra $\mathbb{C}[\tau]/(\tau^2)$, commutative, associative.

So, we use nilpotent τ as a nilpotent memory algebraic unit.

The constraint $\tau^2 = 0$ is the log's **unique-key rule**: the same entry cannot appear twice. If an identical step is attempted a second time, its log entry is silently dropped - the second τ squares to zero, leaving the first τ untouched.

The log is therefore **tamper-evident**: no fact can be duplicated, no fare can be logged twice. This is what makes the Owner's object trustworthy as a record of the pipeline's run.

Under this interpretation `τi` becomes a rendition of the overflow itself, the one that we encountered at every level – when expression of the arithmetic context fails to render the next step. So, `τi` becomes our trash bin of overflow that Owner Monkey will produce.

6.2. MBP as a nested algebra

This let's us create a nesting by harnessing a sort of "algebraic surgery":

Rung:	Equation:	Term dropped:	Monkey:
F5	`a + bi + τ + τi`	-	Tax
F4	`a + bi + τ`	`τi`	Owner
F3	`a + bi`	`τ`	Manager
F2	`a + |b|`	the phase/sign of `b`	Drink-Runner
F1	`a`	`b`	Line-Cook
F0	`[0; 1] flags`	`a`	Flip-Burger

Each rung removes exactly one coordinate axis from the algebra above it, and what remains is still a legitimate algebra - a quotient of the one above. Right or wrong – this can be compiled.

F4 witnesses the lifts, but direction of these lifts is handled only at the overflow rung of F5. This means that Owner Monkey can see change, but can't see where it goes. The whole Ladder travels back down the pipeline and re-traces each step.

6.3. What is a number?

In effect this allows us to define precisely what "number" types we are dealing with:

Rung:	Number type:	Container for:
0	Binary flag	Presence/absence tally
1	Cipher	Counters for tallies
2	Address	Addresses for counters
3	NaN	Cardinal vs Ordinal registers
4	Set	Container for registers
5	Instance	Instance for containers

A number for this table is treated as a signifier for distinction grammar. Binary flag distinguishes if something is present or absent, a cipher is a number that is a counter, an address is what holds different counters on the number line.

The interesting part comes when we consider that when we try to track the ordering of numbers: we have two competing paradigms of cardinal vs ordinal. But what if the numbers allowed to be a bi-sheet structure? Anyway, whatever acts as a container for addresses can't be a number.

6.4. The thing to think about at night

What follows from this is that maybe "a set" *as a distinction* is an interpreter-level (L5) affordance that took the roles from the compiler (L4) and the assembly (L1–L3).

In a way, when we compute with sets it appears as if though we are operating in an instance that is computing with itself. In effect, collapsing every possible distinction of tally, counter, address and register to a single "set". And this level collision may be the very reason integer, float, and heap overflows occur in the first place. Why?

Because the rule of well-foundedness may have been the very thing that ejected the self-similar nesting as a legitimate concept, which in turn caused the inability to recognize the sets as self-nesting objects. After all, how do you prosecute the prosecutor?

7. Tax Monkey

7.1. What the Tax is

The Owner's object is `a + bi + τ`. So, it cannot close. Because it is “3D”, for a brief moment (if only we could freeze that moment with an algebraic surgery) until it becomes the Tax Monkey algebra:

`d = 1 + a + bi + τ + τi`, with `τ² = 0`, `i² = −1`, `iτ = τi`.

From this POV, the `τi` can be treated as a residue of what is left after the nilpotent traces collapse to null. We no longer have the traces (under nilpotency rule a trace of a trace becomes null), but we can trace where the whole system is progressing to. τi emerges from the iteration itself, on a meta-level.

The record of τi is the job of the Tax Monkey.

7.2. The overflow

Each iteration leaves a `τi`. Iterations do not stop. The directions accumulate.

This is where the exceptions gather. The Owner produces one watched field; the Tax produces one `τi` per iteration. As computation deepens, the ledger grows. The Tax Monkey lives in a constant state of induced overflow which is bound to eventually pile-up.

Therefore this presents a very challenging issue for implementation, because the amount of the overflow may theoretically easily reach into Planck-scale volumes.

Therefore, the MBP calls for a storage design that is:

- **Logarithmic** in the pile-up size: storage grows as `log(N)` for `N` iterations, not as `N`. (logarithmic overhead)
- **Self-similar**: the pile-up has structure at every scale. The same shape recurs at each level. A storage design that exploits the self-similarity can compress the pile-up without losing the directions.

And this is where we must go into specifications.

8. Specifications

8.1. Before specs

The exact specifications were laid out in a very lengthy WIP monograph that will be referenced from now on. Before we go into detail, I suggest first to read the standalone papers, that render the monograph a bit more reader-friendly:

1) “Generative Integer Fractal from minimal rules”. A paper about the combinatorial object that the whole architecture was based on:

https://github.com/Bitriada/poetica-mathematica/blob/main/3_Standalone_papers/Arithmetic_Bitriada/Generative_Integer_Fractal_from_minimal_rules.pdf

2) "History_of_Math_as_a_Sequence_of_Cognitive_Revolutions". A paper about what fares did we pay to advance our understanding of numbers: https://github.com/Bitriada/poetica-mathematica/blob/main/3_Standalone_papers/Cognitive_counting/History_of_Math_as_a_Sequence_of_Cognitive_Revolutions.pdf
3) "Generative_Construction_of_6_Geometric_Freedoms". An in-depth case study of how mathematical operations evolve under a restriction of needing geometrical validation: https://github.com/Bitriada/poetica-mathematica/blob/main/3_Standalone_papers/6_geometric_freedoms/Generative_Construction_of_6_Geometric_Freedoms.pdf

Now the relevant objects/structures from the monograph will be listed.

The monograph "**Poetica Mathematica: Cogitatio ex Nihilo et Speculum Scientiae":** https://doi.org/10.5281/zenodo.23698076

8.2. L1-L3

- **Fare Principle** - (Ch. II, §2.2). The principle behind the need to separate semantic layers.
- **Genesis Spiral** (Ch. I, §1.2) - binary seed and epoch progression;
- **Arithmetic Bitriada** (Ch. I, §1.6) - the 117-point lattice a discrete fractal solution for scaling/data storage and the basis for the MBP pipeline.
- **Tryptich Engine** (Ch. III, §3.3) - a candidate compilation of the operations/instances.
- **Fare Ladder** (Ch. III, §3.5) - algebraic atlas (F0–F5).

8.3. L4

- **UC Algebra** (Ch. IV, §4.2.1) - 4-dimensional ultra-complex algebra with nilpotent trace $\tau^2 = 0$; the Manager's complex field.
- **Fare Algebra** (Ch. IV, §4.1.1) - prime-indexed nilpotent algebra encoding square-free basis. Needed to bootstrap traces into complex field.
- **UC1–UC3** (Ch. III, §3.4.2) - perspectival ultra-complex algebras (Complex Point, Complex Line, Complex Disc).
- **UC4** (Ch. III, §3.4.3) - loop completion of perspectival algebras; closed complex field with nilpotent holonomy τ.

8.4. L5

- **Fare–Riemann 3-sphere** (Ch. IV, §4.4) - unit sphere S^3_FR of UC; continuous geometric stage for L1–L3 as complex objects. Candidate geometric solution for L5.
- **Tribit** (Ch. IX, §9.1.1) - the computational primitive $\Gamma = a + bi + \tau c + \tau di$; an element of UC carrying value and trace, processed by the TriPU and navigated on the Dimensoid.
- **C_8 action on S^3_FR** (Ch. IV, §4.4.3) - eight rotation operators O_k providing discrete rotational symmetry. The operational set on data structures of L5.
- **Dimensoid** (Ch. V, §5.1) - geometric facet of the Algebroid–Dimensoid–Tribit triad; the sphere (S^2 at 1DB, S^5 at 3DB, S^7 at 5DB) carrying the discrete lattice.
- **Spectator's Nest** (Ch. IV, §4.5.3) - balance viewpoint on 3-sphere, tribit and dimensoid.
- **Ouroboros Sphere** (Ch. VI, §6.3) - closed sphere binding the 117-point lattice with the ghost disc; discrete geometric realization. Another candidate rendition.
- **TriPU** (Ch. IX, §9.1) - Tribit Processing Unit; computational architecture (C, Q, Γ cores + fourth coherence core) navigating the Algebroid on the Dimensoid.
- **UC-Quaternions** (Ch. VI, §6.2) - 16-dimensional algebra encoding the 4×4 zeta-array; supplies disentangling map between quaternionic strands. Candidate for providing weights in parallel computing of the MBP
- **Cardinal Mold** (Ch. VIII, §8.6) - universal six-rung ladder (Null → Void → Binary → Number → NaN → Set → Zero) generalizing the Fare Ladder; structural template.

8.5. L6

- **Atlas Cube as 16-vertex tesseract** (Ch. IV, §4.4.6) - 4-dimensional hypercube containing the Fare–Riemann sphere; discrete coordinate frame.
- **Ghost Disc** (Ch. IV, §4.2.4.4; Ch. V, §5.1) - nilpotent trace membrane $D^2_\tau = \{\tau s : |s| \leq 1\}$; invisible binding surface of the Dimensoid and audit memory.
- **Maiden Math** (Ch. VI, §6.4) - carry operator $\Lambda = \tau^3$, mirror algebra, three-pass gate; mechanism for completing spectral trace and enforcing ledger.
- **Coherence Gate** (Ch. VII, §7.5) - four-protocol gate (Letha, Ouroboros, Apocalypsis, Calypso) judging systemic coherence; primary arbitration instrument.

- **Transit-Fare Algebra TFA** (Ch. IV, §4.3.5) - 12-dimensional algebra with generators i, τ, θ ($\theta^3 = 8\theta$); encodes tiered Fare structure and carry $\Lambda = \tau\theta^2$.
- **Bubble Wall BW_3** (Ch. V, §5.5.7.4.3) - enriched morphology algebra with three walls and currents; interfaces between trace, time, and evolutionary axes.
- **Zeta-string** (Ch. VI, §6.1) - binary encoding of the critical line fracturing into the 4×4 zeta-array; spectral signature ledger.
- **Trinumeral** (Ch. VIII, §8.4) - three-base (binary, integer, float) counting structure with graded nilpotent trace; heterobase ledger primitive.
- **Heterobase Summation Principle** (Ch. VIII, §8.3) - generative principle that holding two incommensurable bases without reduction seeds a new base; mechanism for expanding the ledger without loss.
- **Spectator Math** - Chapter VI, §6.3a "Spectator Mathematics" (pp. 228–240), subsections §6.3a.1–§6.3a.14; core results: Definition 6.3a.6 (Six Spectator Positions), Proposition 6.3a.2 (Spectator Invariant Table), Definition 6.3a.5 (Spectator as Stabilizer), Theorem 6.3.1 (No-Choice Spectator), Definition 6.3.2 (The Spectator).
- **Quantum Spectator** - Book II, Part II, "Spectator Invariants in the T_8 Algebra - The Quantum-Cognitive Bridge" (Addendum XXIV), §I–§XII; core results: §III Theorem (Spectator Invariant Position), §V Theorem (Spectator Dynamics) and Definition (Spectator State), §VIII Definition (T_8-Meta Algebra), §X Theorem (Spectator Nest).
- **Spectators and attractors:** Book II, Part I - "Quantum Dynamics as Attractor Topology" - §VII "The Observer in the System", esp. §VII.3 "The Spectator as Point on Attractor" (table mapping Unfathoming / Deep / Backward / Forward / Observer / Facilitator to positions on the attractor manifold).

9. Conclusion

9.1. The Fare Principle as a Design Law

The Monkey Business Pipeline is not a stack of programs; it is a ledger of sacrifices. The Fare Principle ("To gain computational power, sacrifice a form of certainty") is the generative engine of the entire architecture. L1 surrenders the continuum. L2 surrenders division. L3 surrenders sign and phase. L4 surrenders execution. L5 surrenders direction. L6 surrenders order. The architecture is the physical manifestation of these paid fares, marked indelibly by the nilpotent trace.

9.2. Security through Algebraic Incapability

The primary innovation of the MBP is its security model. It does not rely on sandboxing, serialization formats, or runtime checks. It relies on **algebraic incapacity**. A layer cannot leak into another layer because it literally lacks the vocabulary to express the other layer's operations. The L1 Flip-Burger cannot execute integer multiplication because it is not in its moveset; the L4 Manager cannot re-execute the workers' math because the mapping is injective but non-multiplicative. Conventions fail; algebras hold.

9.3. The Central Inversion

The structure of the MBP inverts the traditional hierarchy of computational privilege. In standard systems, higher levels have more power and agency. In the MBP, **the environment with the widest view has the narrowest agency**. The workers see only their immediate task and perform complex arithmetic. The Manager sees the whole complex field but can only weigh it. The Owner sees the entire run but can only watch. The Tax Monkey sees the whole history but can only record. Power is inversely proportional to perspective.

9.4. Spatialization, not precision

The word-size ladder 8 → 16 → 32 → 64 → 128 → 256 is historically a ladder of numerical precision. In the MBP it indexes something else: **nesting depth and tensor rank**. The progression is dimensional, not numerical: a 1D tape, a 2D card, a 3D loom, a complex plane, a 3-sphere, a tesseract. Each rung adds one *dimension of context* - one more way for a value to be addressed, held, and audited - without widening the underlying 8-bit string. The architecture is a spatialization of computation: the register stays the same size, and what grows is the space the register's contents live in. This is why the pipeline can call itself minimal and still reach six levels: it is not stacking precision, it is stacking *addresses*.

9.5. The package as normal form

The 1→2→4 turn progression and the 6→12→24 operation progression are the doubling signature of the hyperoperation sequence. Each level retains the previous level's operations under a new index - `+1/-1` becomes `+a/-a`, which becomes `+b/-b` - and unlocks the next pair: addition, then multiplication, then exponentiation. Read structurally, this is a claim about closure: **each level's terminal operation is the closure of the operation the level above uses to open its register, and the package it hands up is a normal form at that level.** The claim is strongest where the algebra is most explicit - at L5–L6, $\tau^2 = 0$ is

literally a normalization rule, and a duplicate fare is annihilated - and weakest at L2, where the card is a matrix rather than a canonical form. But this is a structural analogy until proven without doubt.

9.6. Type confusion as level collision

The ontological ladder - flag → cipher → address → NaN → set → instance - names the different kinds of things a number can refer to. In §6.4 we conjectured that "set" is an L5 interpreter affordance that collapses lower-level distinctions into a single container. Read plainly, that conjecture is adjacent to the theory of **level collision**.

If we look at common memory-safety and numerical bugs through this lens, they resemble collisions between ontological rungs: a pointer (an address, L2's structure) read as an integer (a counter, L2's object); a cardinal read as an ordinal; a container read as a value; an integer read at float saturation.

The MBP does not claim to solve type confusion in general. Rather, it offers an architectural response: instead of relying on runtime type checks or boundary conventions to catch these collisions, the pipeline enforces algebraic incapacity. Because the levels do not share a representation, the boundary where the wrong operation could cross is eliminated. Whether this structural property can scale to address broader classes of software vulnerabilities remains an open question, but the invariant - never let levels share a representation - presents a distinct alternative to convention-based type safety.

9.7. The Fare Ladder as a Universal Audit Trail

Because every layer pays a fare, the nilpotent trace acts as an inherent audit log. You cannot double-count a step because the algebra refuses to duplicate a record. The system achieves "reproducibility by construction" - not because you replay the code, but because the mathematics itself guarantees that the same rung, replayed with the same counter, is the same instance. This offers a blueprint for differentially debugging long-latency anomalies by looking at the exact rung where the trace fractured.

9.8. The Algebraic Refactoring of Layered Systems

The Monkey Business Pipeline proposes that security, auditability, and reproducibility can all be derived from the same algebraic source: a nested quotient structure where each level’s incapability is its security guarantee. The MBP is an attempt at an algebraic refactoring of the entire layered-system problem.

Propositions:

A new security model. "Conventions fail; algebras hold." We replace sandboxing, type checks, and runtime enforcement with algebraic incapacity. A level cannot leak into another because it lacks the vocabulary to express the other's operations.

Reproducibility by construction. Through the nilpotent trace, replay becomes unnecessary. The algebra itself guarantees that the same rung, traversed with the same counter, is the same instance. You do not verify the run; the algebra dictates it.

A unified ontology of numerical computation. From binary flags to sets and instances, the Fare Ladder provides a single ontological spine. Type confusion is reinterpreted as "level collision" between ontological rungs - a structural diagnosis of where modern systems break.

An inversion of computational privilege. The structure inverts the traditional hierarchy of power. The widest perspective (the Tax Monkey) has the narrowest agency. The more an environment sees, the less it is allowed to do.

9.9. An Invitation to the Build

The MBP is presented as a conjectural framework. It is not a finished software library, but a structural specification. The open items (the exact quotients for sign/phase, the projection of UC4, the prime angles of the 117-point lattice) are not bugs; they are the exact coordinates for the first test build. The architecture is ready to be instantiated, whether as a software framework, a heterogeneous offload system, or a theoretical model for quantum-classical boundaries.

So, if we say built this machine. How it would be called?

Here are some options: Arithmeclast, Elenchoscope, Mathematical Athanor – use whichever you like, and we'll see which sticks.

Source boundary:

This paper is a standalone synthesis of the schema design and generative architecture presented in Kirill Kovlekov, *Poetica Mathematica: Cogitatio ex Nihilo et Speculum Scientiae* (Version 0.25). The framework of the six-level environment (L1–L6), the triadic scale indexing (osi), and the Fare Ladder (F0–F5) follow the monograph's internal sequence, chiefly the Tryptich Engine extrinsic architecture in Chapter III (§3.3, covering the algebraic, geometric, and spectral axes), the Fare Ladder (Ch. III, §3.5), and the Ultra-Complex algebras in Chapter IV (§4.1–4.4). The geometric progression and the N-Z-Q domain advancements of the Worker Monkeys (L1–L3) are reconstructed here using the

framework of the *Generative Construction of 6 Geometric Freedoms* paper, specifically the operational ladder and the identification of the Apeirogon as the Gödelian threshold. The combinatorial progression of turns and operations (1-2-4 and 6-12-24) and the 117-point lattice referenced in the specifications are drawn from the *Generative Integer Fractal from minimal rules* paper.

The transition from calculational (L1–L3) to computational (L4–L6) environments, the Central Inversion (where the widest perspective has the narrowest agency), and the algebraic refactoring of layered-system security are presented here as a self-contained conjectural framework. The session-type reading and the Turing/Babbage/Jacquard narrative frame are presented as structural analogies, not as formal typing judgments or claims of physical embedding. The identification of memory-safety bugs as "level collisions" is offered as a structural diagnostic lens suggested by the ontological ladder, not as a formal proof that the MBP eliminates all software vulnerabilities. The full monograph is available via Zenodo at https://doi.org/10.5281/zenodo.23698076.